



Prescribe

Software Requirements Specification (SRS) - PRESCRIBE

1. Introduction

[1.1 System Overview](#)

Prescribe is a secure, cloud-based prescription management platform. It digitizes the medical prescription lifecycle, allowing Doctors to issue structured records, Patients to access and share their medical history, and Admins to manage the ecosystem. The system is architected using a React frontend and a Spring Boot microservices-aligned backend, hosted on AWS.

[1.2 Key System Features](#)

- **Role-Based Access Control (RBAC):** Strict permission levels for Doctors, Patients, and Admins.
- **Secure Authentication:** JWT-based login with password hashing via Spring Security.
- **Digital Prescription Management:** Full CRUD (Create, Read, Update, Revoke) capabilities.
- **Centralized Cloud Storage:** Structured data via AWS RDS and document storage via AWS S3.
- **Audit Logging:** Traceability of all sensitive system actions including record views and data exports.

2. User Roles and Characteristics

2.1 Doctor

- Role: Authorized medical professional responsible for issuing prescriptions.
- Capabilities: Create, edit, and revoke prescriptions; view history of assigned patients; upload digital copies of prescriptions to cloud storage.
- Constraint: Requires verified registration credentials.

2.2 Patient

- Role: End-user and owner of the medical record.
- Capabilities: View personal history, search/filter prescriptions, and view/download prescription documents stored in the cloud.
- Constraint: Cannot modify or delete any prescription data.

2.3 Admin

- Role: System monitor and manager.
 - Capabilities: Manage user accounts, verify Doctor licenses, monitor audit logs, and maintain system integrity.
-

3. Functional Requirements

3.1 User Management (UM)

- UM-01 Authentication: Secure login via Spring Security using JWT. Tokens must include role-based claims.
- UM-02 MFA (Optional/Future): Capability to integrate Multi-Factor Authentication for Doctor and Admin accounts.
- UM-03 Compliance Features: Right to be Forgotten: Soft-deletion of user profiles where medical data is

pseudonymized for legal retention but removed from active access.

Data Portability: Ability for Patients to export their entire history in CSV or structured JSON format.

3.2 Prescription Management (PM) [P0]

- PM-01 Digital Entry: Doctors shall input: Patient_ID.
- PM-02 Document Upload: Direct multi-part file upload to AWS S3. Files must be scanned for malware upon upload (optional/backend hook).
- PM-03 Versioning: Every update to a prescription triggers a new version in the Prescription_Versions table. The Prescription table points to the Current_Version_ID.
- PM-04 Revocation: A "Revoke" action requires a reason code (e.g., Medication Change, Error, Patient Request) and is irreversible.

3.3 Collaboration & Sharing (CL)

- CL-01 Patient Dashboard: Real-time updates using WebSockets or polling to notify patients of new prescriptions.
- CL-02 Secure Sharing: Generation of AWS S3 Pre-signed URLs with a maximum TTL (Time To Live) of 24 hours.

3.4 Audit & Logging

- AL-01 Access Logging: Log entries must include: Timestamp, User_ID, Action_Type, Resource_ID, Status, and Request_Origin_IP.
- AL-02 Immutable Logs: Audit logs should be stored in a write-once-read-many (WORM) compliant manner or secondary secure database.

4. Non-Functional Requirements

4.1 Performance & Scalability

- NFR-01 (TTI): Dashboard Time to Interactive < 3 seconds on a standard 4G connection.
- NFR-02 (Latency): Backend API Latency < 200ms for 95th percentile of standard CRUD operations.
- NFR-03 (Scalability): Stateless backend to support horizontal scaling via AWS Auto Scaling Groups.

4.2 Security & Compliance (Critical)

- NFR-04 (Access): Method-level security in Spring Boot using @PreAuthorize based on JWT roles.
- NFR-05 (Privacy): PII (Personally Identifiable Information) must be masked in logs.

4.3 Availability & Reliability

- Target Uptime: 99.5% monthly availability.
 - Database Reliability: Use of Multi-AZ deployment for AWS RDS to ensure failover capability.
 - Atomic Transactions: JPA @Transactional usage to ensure data consistency during complex multi-table writes.
-

5. Data Model & Storage

5.1 Primary Entities

- User: UUID, Email, Password_Hash, Role, Created_At, Status.
- Patient_Profile: User_ID, DOB, Blood_Group, Address.
- Prescription: ID, Patient_ID, Doctor_ID, Status, Current_Version_ID.
- Prescription_Version: ID, Prescription_ID, Medication_Data (JSONB), S3_Doc_Path, Change_Log, Timestamp.

5.2 Storage Strategy

- Relational Data: AWS RDS (PostgreSQL) for transactional data and versioning.
 - Blob Storage: AWS S3 for prescription images and medical documents.
 - Caching: Redis (AWS ElastiCache) for session management and frequent lookups (Optional).
-

6. Disaster Recovery & Backup Strategy

- Backup Frequency: Daily: Automated incremental database backups with 30-day retention.
Weekly: Full system snapshots stored in a separate AWS region (Retention: 1 year).
 - RTO (Recovery Time Objective): Operations restored within 4 hours of catastrophic failure.
 - RPO (Recovery Point Objective): Maximum data loss tolerance of 24 hours (last daily backup).
-

7. Design Constraints & Deliverables

- Stack: React.js (Frontend), Java Spring Boot (Backend), AWS RDS, AWS S3.
 - Deployment: Dockerized containers orchestrated via AWS ECS (Elastic Container Service).
 - Deliverables: Source code in a private repository.
 - Docker Compose for local development.
 - CI/CD Pipeline (GitHub Actions) for automated testing and deployment.
 - Swagger/OpenAPI documentation for all REST endpoints.
 - Admin and User manuals.
-

8. Success Metrics

- User Efficiency: 30% reduction in time spent on patient file organization compared to manual/paper-based systems.
 - System Reliability: Zero data loss incidents reported over a 12-month period.
-

9. Risks & Assumptions

9.1 Assumptions

- Medical professionals possess a valid license and will be vetted manually during the registration phase.
- End-users have a minimum internet bandwidth of 10 Mbps for uploading/viewing imaging files.

9.2 Risks & Mitigation

- Storage Costs: High-resolution uploads may escalate costs.
Mitigation: Implement file size limits (50MB) and S3 Intelligent-Tiering to transition older files to cold storage.

- **Data Breach:** Unauthorized access to PHI. *Mitigation: Regular penetration testing, strict RBAC, and mandatory encryption of all PII.*
- **Regulatory Shift:** Changing DPDP Act, 2023 rules. *Mitigation: Modular architecture for easier updates to compliance logic.*